

EIE2111 GUI Programming

Dr. Lawrence Cheung

Richard Pang

Semester 2, 2021/22

Topics

- Introduction to C#
- Some Simple GUI programs
- C# with C++
- Read/Write Text Files
- Multiple Forms
- GDI (Windows Graphical Device Interface)

Introduction to C#

- A message from Microsoft official web site:

MS removed the Windows Forms C++/CLI template starting in either VS2012 or VS2013.

The official explanation is that they want to encourage managed desktop UI development in C# or VB and relegate C++/CLI to serve as glue between native back end code and managed UI code.

Introduction to C#

- Thus now we are going to learn C# for GUI (Windows Forms) applications.
 - GUI = Graphical User Interface
- The graphical user interface (GUI), is a type of user interface that allows users to interact with electronic devices through graphical icons and visual indicators such as secondary notation, instead of text-based user interfaces, typed command labels or text navigation.
 - From Wikipedia.

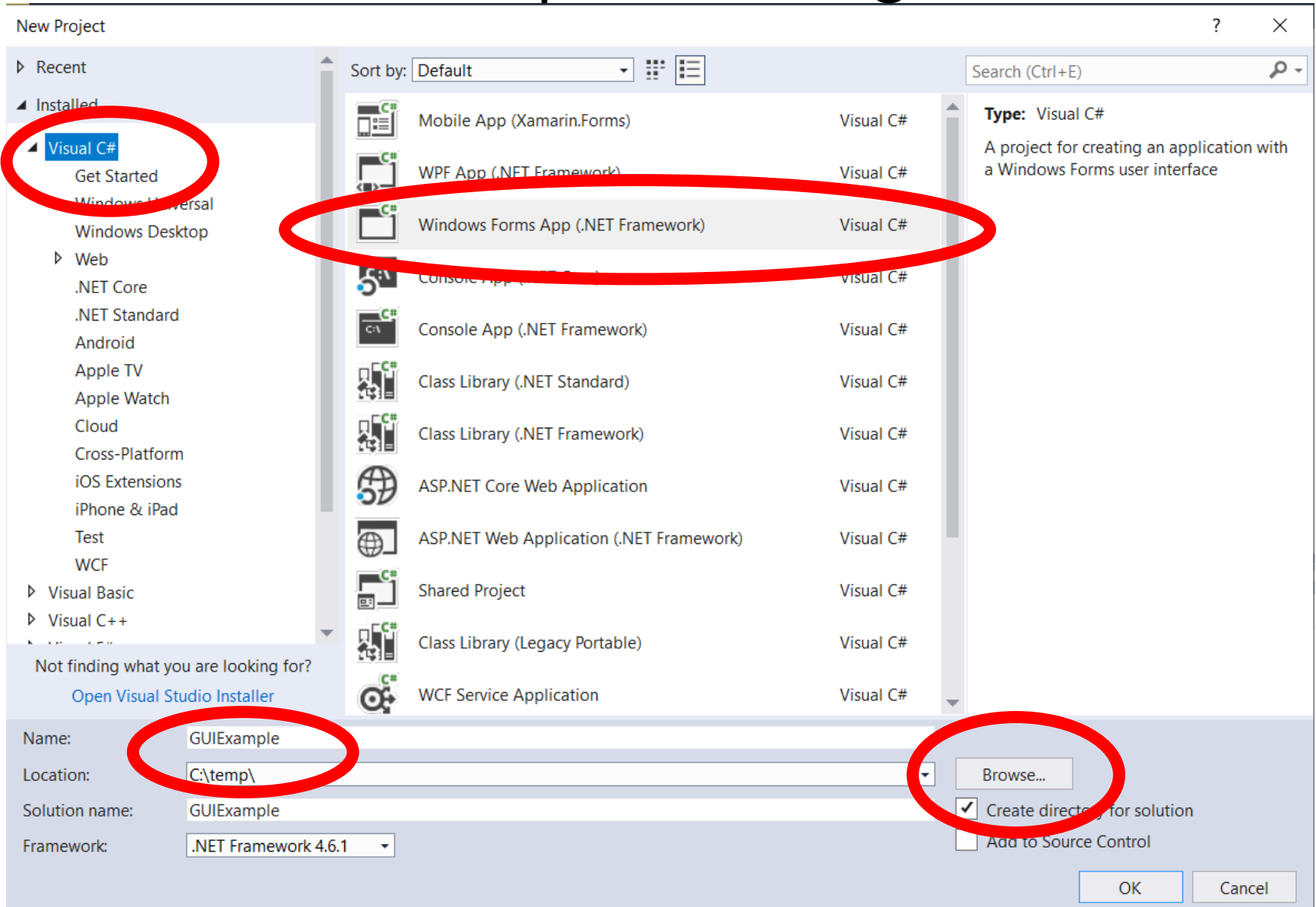
Introduction to C#

- C# is a general-purpose, object-oriented programming language.
- It is very similar to C++ but better than C++
 - C# is a pure object-oriented language.
 - C# has native garbage-collection.
 - It easily makes GUI in C#.
 - C# has a huge standard library.
- Most of C++ statements can be applied in C# but C# provides type safety.

Some Simple GUI Programs

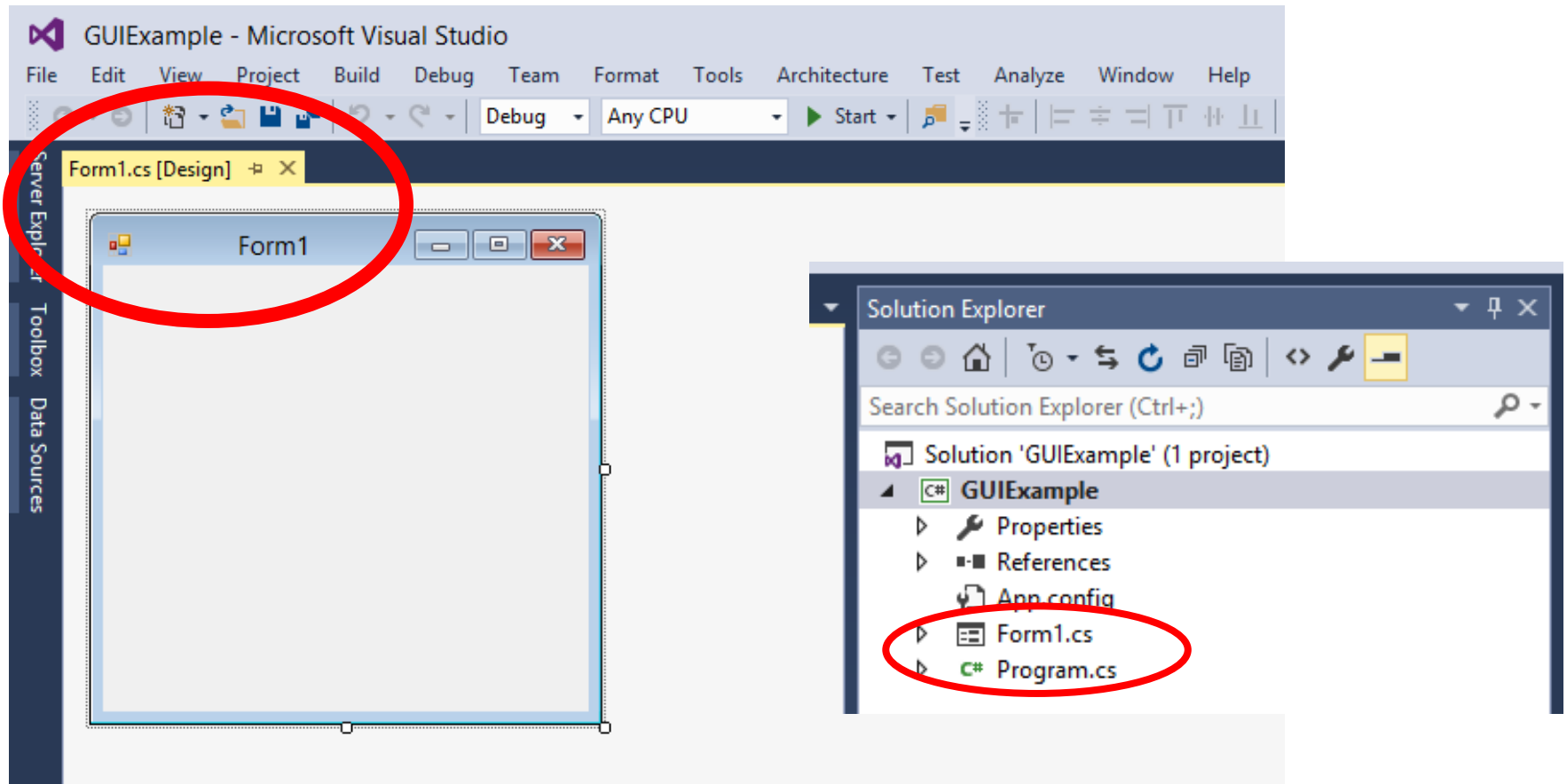
- Example 1: Hello World
 - Step 1: Create a new form.
 - Main Menu $\Rightarrow$ “File” $\Rightarrow$ “New” $\Rightarrow$ “Project”.
 - Select “Visual C#”, “Windows Forms Application”.
 - Type in project name “GUIExample”.
 - Click “OK”.

Some Simple GUI Programs



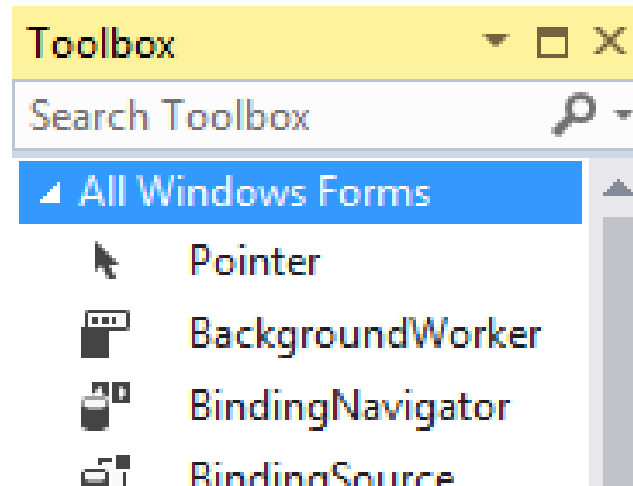
Some Simple GUI Programs

- A form called “Form1.cs” is created with “Program.cs”.
 - “cs” means “C#” (s = sharp).



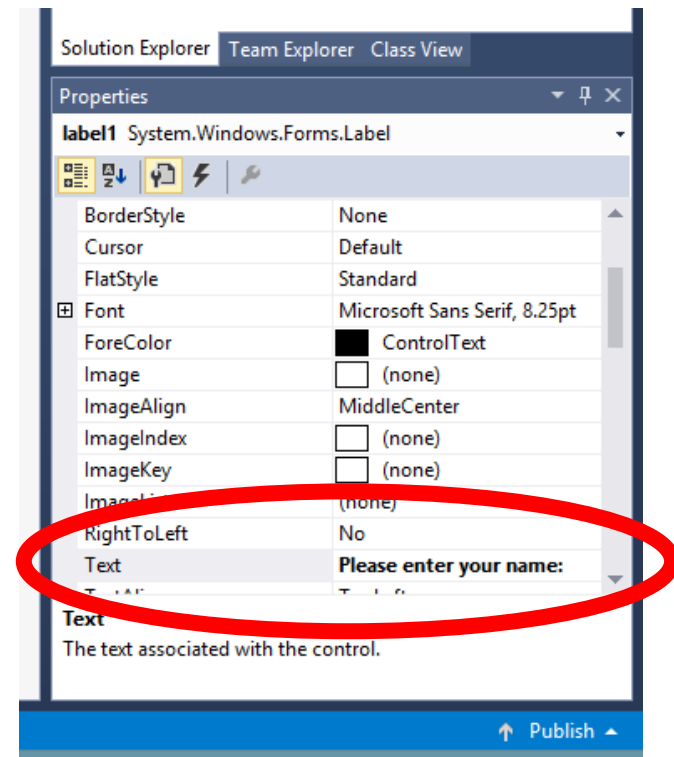
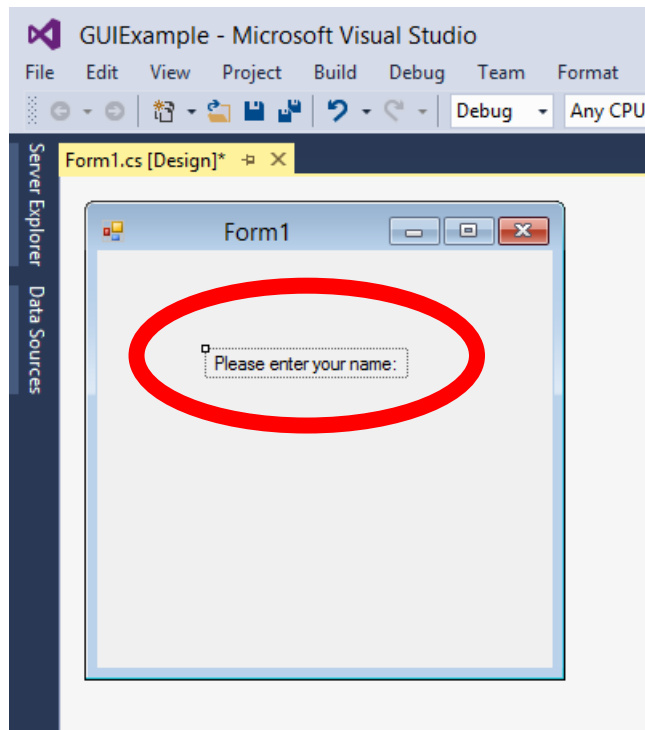
Some Simple GUI Programs

- Step 2: Add some GUI components on the form.
 - Main Menu $\Rightarrow$ “View” $\Rightarrow$ “Toolbox”.
 - Put the “Toolbox” menu on the right hand side.
 - Click “All Windows Forms”.



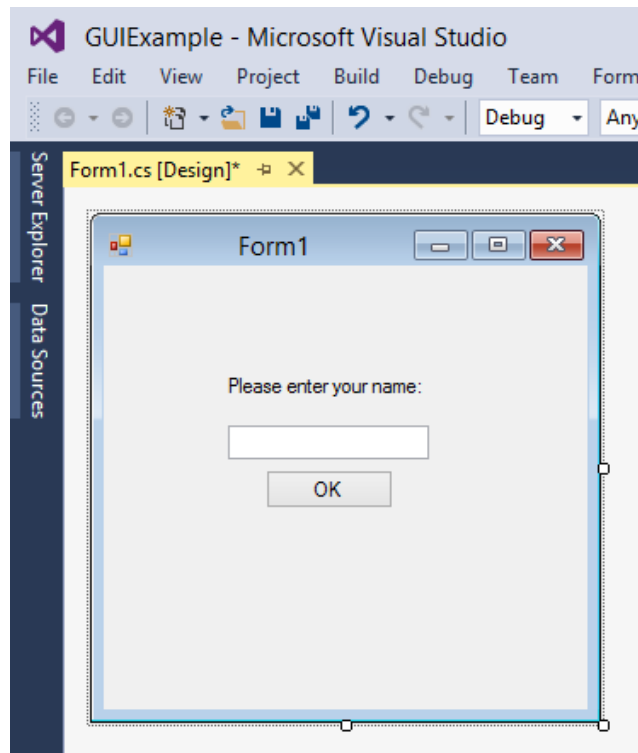
Some Simple GUI Programs

- Drag and drop “Label” from “Toolbox” menu.
- Go to click the label once. Then you should find its “Properties” window on the lower right corner.
- Go to “Text” field and enter “Please enter your name:”.



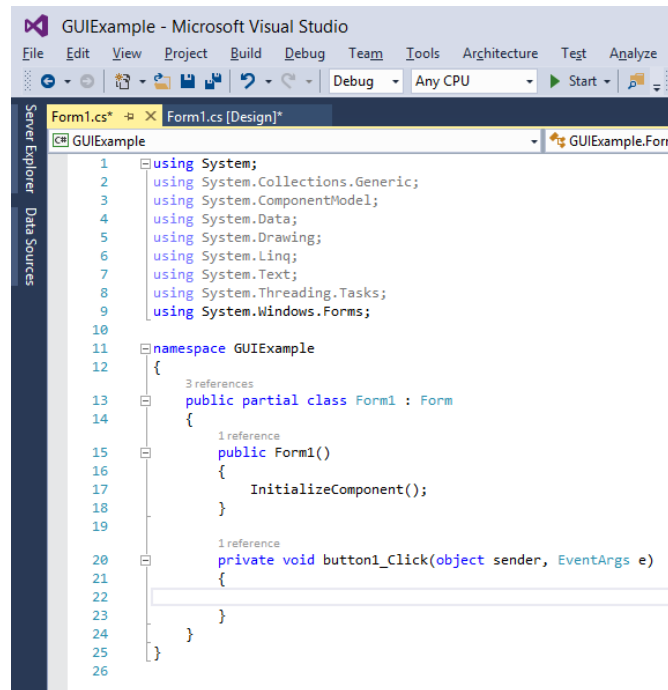
Some Simple GUI Programs

- Use the similar way to add a text box and a button to the form.
 - The length of the text box is same as the label.
 - The text of the button is “OK”.



Some Simple GUI Programs

- Step 3: Create an event handler.
 - Double **click** the **button** and the file “Form1.cs” is opened.
 - One is the outlook (“Form1.cs [Design]”).
 - One is the program code.



```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 namespace GUIExample
12 {
13     3 references
14     public partial class Form1 : Form
15     {
16         1 reference
17         public Form1()
18         {
19             InitializeComponent();
20         }
21
22         1 reference
23         private void button1_Click(object sender, EventArgs e)
24         {
25
26     }
```

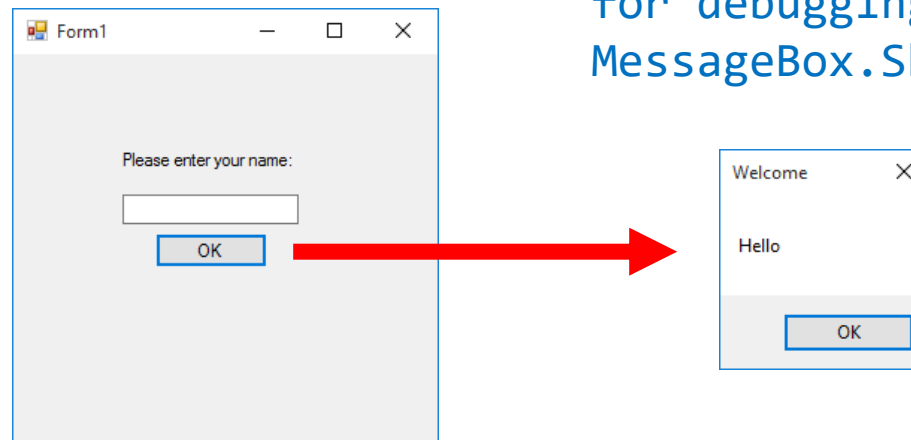
Some Simple GUI Programs

- Add the following program code into the function **button1_Click**:

```
string message = "Hello ";  
string title = "Welcome";  
MessageBox.Show(message, title, MessageBoxButtons.OK);
```

- Then compile and run the program.
 - Click the button “OK”.

Tips: Use MessageBox
for debugging:
`MessageBox.Show(message);`



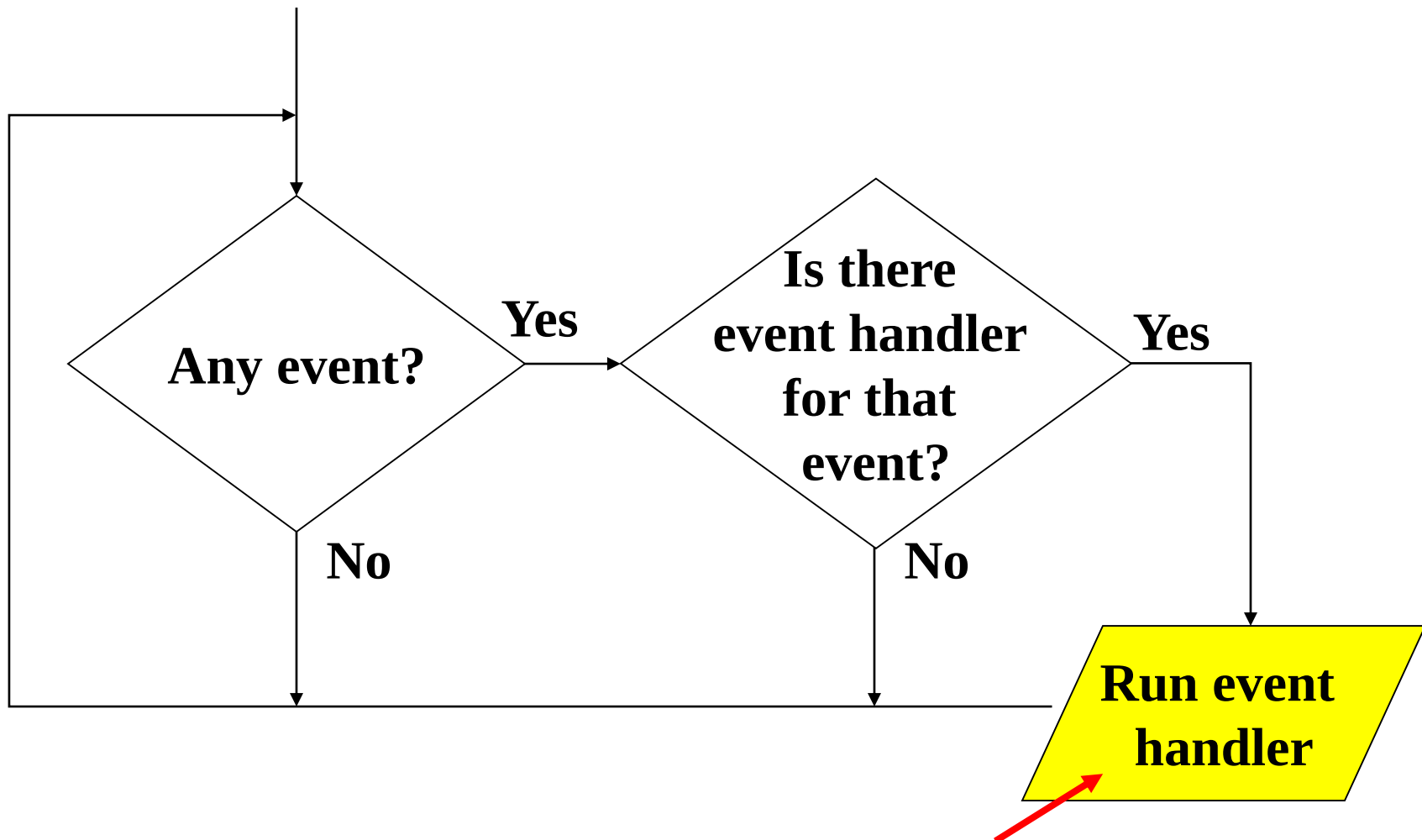
Some Simple GUI Programs

- Writing Event Handler
 - A major task that a GUI designer needs to do is to determine what will happen when a GUI is invoked.
 - Every GUI component may generate different kind of “events” when a user makes access to it using his/her mouse or keyboard.
 - Example 1: if a user moves his/her mouse on top of a button, then an event of that button will be generated to the Windows system.
 - Example 2: if a user further clicks, then another event of that button will be generated (actually it is the click event).

Some Simple GUI Programs

- For any event generated, the system will first check if there is an event handler, which defines the action for that event.
- As a GUI designer, he/she needs to develop the event handler to determine the action that he/she wants Windows to take for that event.

Some Simple GUI Programs



The part that a GUI designer needs to develop

Some Simple GUI Programs

```
MessageBox.Show(message, title,  
                MessageBoxButtons.OK) ;
```

- Display a message box
 - message = a string to display inside the message box
 - title = a string as the title of the message box
 - MessageBoxButtons.OK = message box button
 - » Other choices: AbortRetryIgnore, OKCancel, RetryCancel, YesNo and YesNoCancel

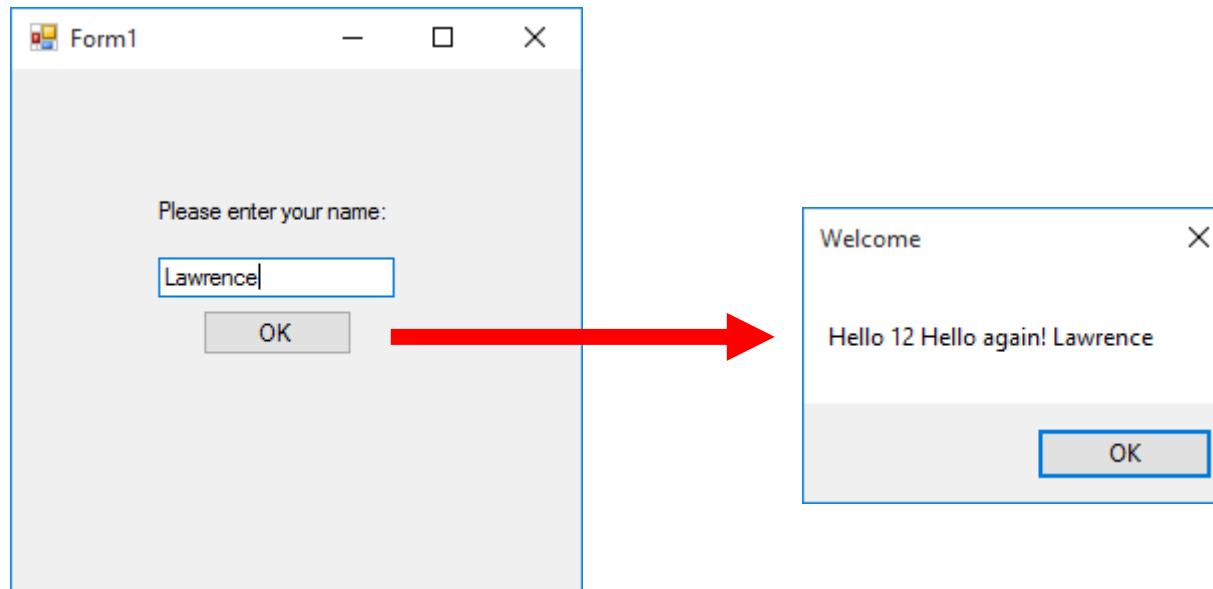
Some Simple GUI Programs

- Example 2: Receive an input from the text box and display it.
 - Modify the program code into the function **button1_Click**.
 - `textBox1.Text` is the variable holding the string in the text box.

```
string message = "Hello ";
string title = "Welcome";
int a = 12;
string c = " Hello again! ";
MessageBox.Show(message + a + c + textBox1.Text, title,
MessageBoxButtons.OK);
```

Some Simple GUI Programs

- Then compile and run the program.

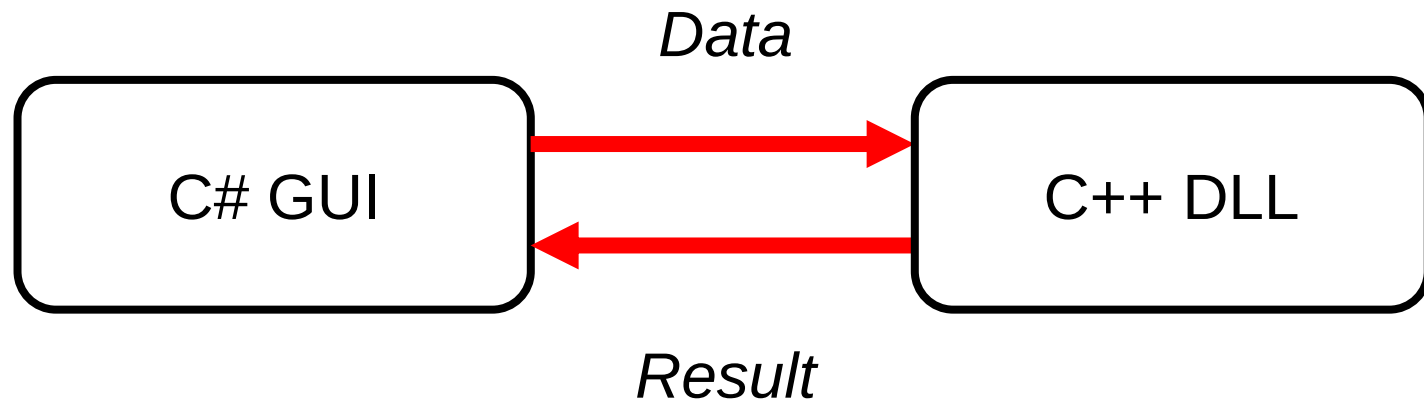


C# with C++

- C# is managed code
 - It runs under a Common Language Runtime (CLR) environment.
 - It cannot be accessed outside the runtime environment.
 - Advantage: avoid many typical programming mistakes.
- C++ is unmanaged code
 - It compiles straight to machine code.
 - It can be directly executed by the Operating System.
 - Advantage: you can run the same code on different computers by recompiling it.

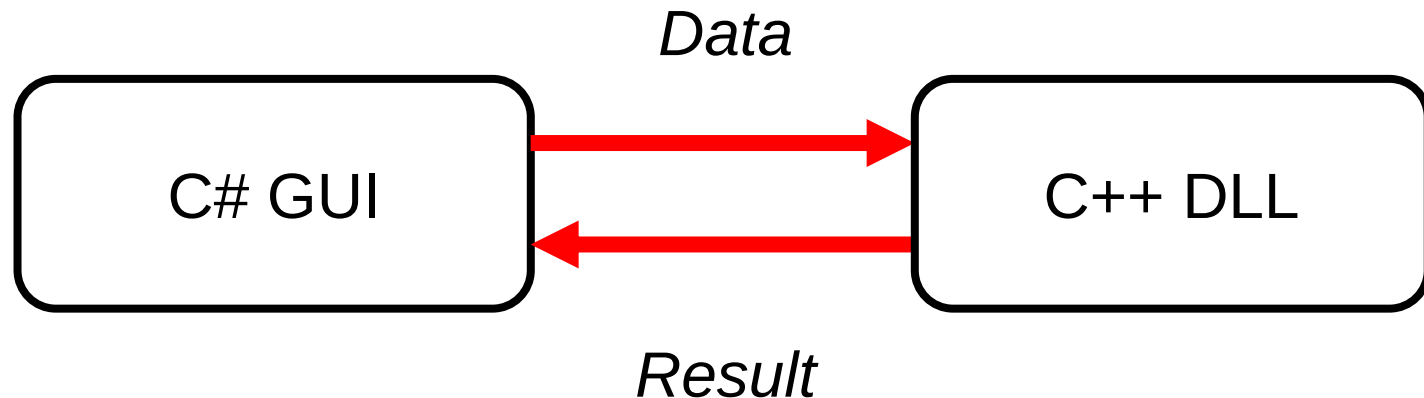
C# with C++

- Is it possible to make use of C++ programs in C# GUI program? Yes!
- Example 3: Use a C# GUI program to get two integers, add them together with an object by using C++ DLL (Dynamic Link Library) and show the result through the GUI program.



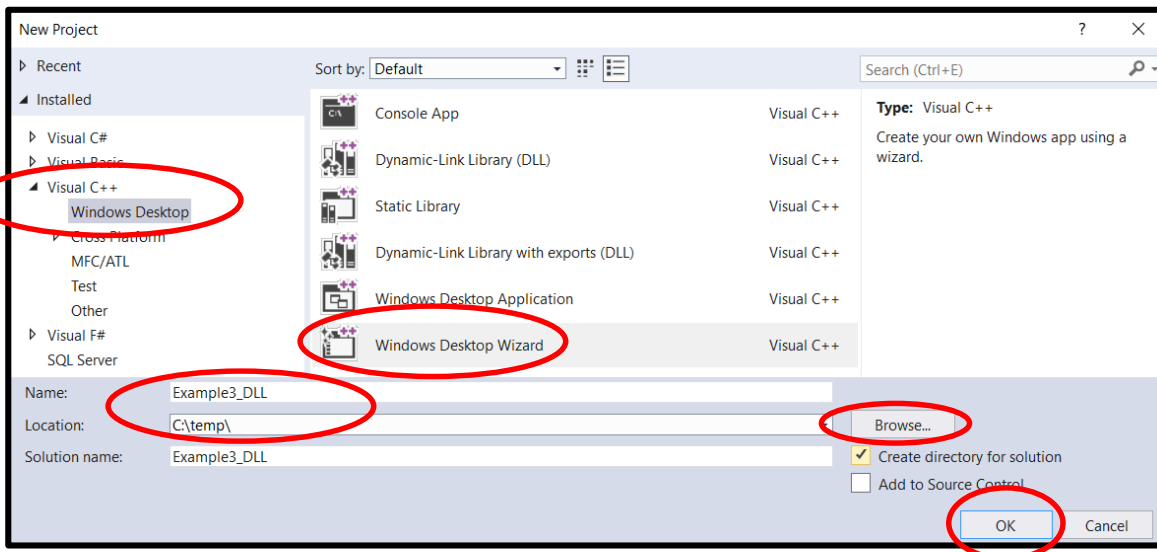
C# with C++

- DLL (Dynamic Link Library)
 - It acts as a shared object in which multiple programs can call the same DLL more than once.
 - Advantages: save space, extensive code reuse



C# with C++

- Example 3: Add two integers and show the sum.
 - Step 1: Create a new project for DLL.
 - Main Menu ⇒ “File” ⇒ “New” ⇒ “Project”.
 - Select “Visual C++”, “Windows Desktop”, “Windows Desktop Wizard”.
 - Type in project name “Example3_DLL”. Click “OK”.
 - Choose Dynamic Link Library. Click “Empty Project” & “OK”.



Windows Desktop P

Application type

Dynamic Link Library (.dll)

Additional options:

☒ Empty project

☐ Precompiled header

☐ Export symbols

☐ MFC headers

C# with C++

- Step 2: Create three files: Example3_Class.h, Example3_Class.cpp and Example3_DLL.cpp

- Example3_Class.h

```
#pragma once
class Example3_Class
{
public:
    Example3_Class(int);
    int getA();
private:
    int a;
};
```

C# with C++

- Example3_Class.cpp

```
#include "Example3_Class.h"
```

```
Example3_Class::Example3_Class(int b)
{
    a = b;
}
```

```
int Example3_Class::getA()
{
    return a;
}
```

C# with C++

- Example3_DLL.cpp

```
#include "Example3_Class.h"

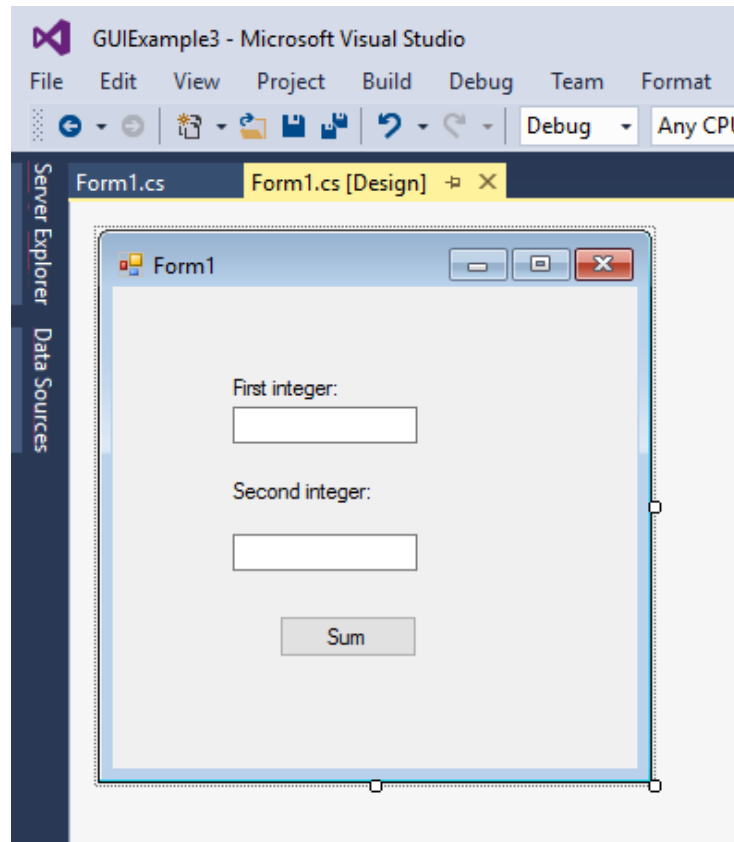
extern "C"
{
    __declspec(dllexport) int doSomething(int x, int y)
    {
        Example3_Class obj(20);
        return x + y + obj.getA();
    }
}
```

C# with C++

- Step 3: Compile DLL (build the solution) but do NOT execute it. It cannot be executed in this way.
 - In the folder Debug, you can find the DLL file Example3_DLL.dll.

C# with C++

- Step 4: Create a C# GUI program named “GUIExample3” and create a form like this:

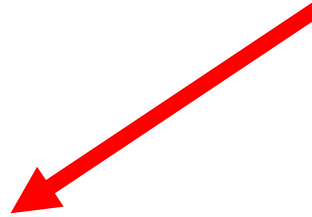


C# with C++

- Step 5: Write the following program code in Form1.cs.

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Drawing;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows.Forms;  
  
using System.Runtime.InteropServices;
```

New program code



C# with C++

```
namespace GUIExample3  
{
```

```
    public partial class Form1 : Form  
    {
```

```
        [DllImport("Example3_DLL.dll")]
```

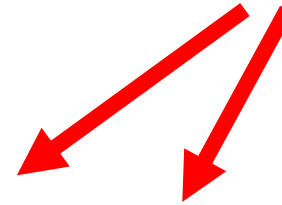
```
        public static extern int doSomething(int a, int b);
```

```
        public Form1()  
        {
```

```
            InitializeComponent();
```

```
        }
```

New program code



C# with C++

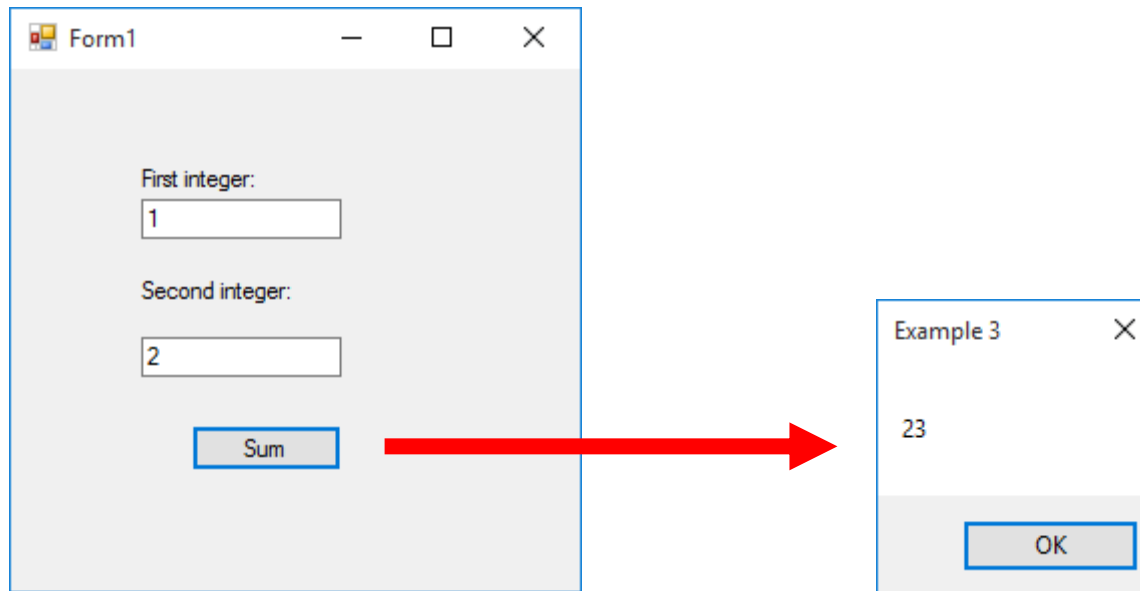
```
private void button1_Click(object sender, EventArgs e)
{
    string string1 = "" +
        doSomething(Int32.Parse(textBox1.Text),
        Int32.Parse(textBox2.Text));
    MessageBox.Show(string1, "Example 3",
        MessageBoxButtons.OK);
}
}
```



New
program
code

C# with C++

- Step 6: Copy Example3_DLL.dll into the sub folder (bin/Debug) of the project “GUIExample3”. Add it from Solution Explorer.
- Step 7: Compile and run the program.



C# with C++

`extern "C"`

- A storage class defines the scope (visibility) and life-time of variables and/or functions within a C program.
- There are four types
 - auto: default storage class, for all local variables

```
{  
    int a;  
    auto int b;  
}
```

- register: also for local variables but they should be stored into a register (a storage in a CPU) instead of RAM

```
{  
    register int a;  
}
```

C# with C++

- static: existing during the life-time of the program

```
static int count = 5;
```

- extern: it gives a reference of a global reference that is visible to ALL program files.

```
extern void write_extern();
```

- extern “C” means it is recognized by a C++ compiler and to notify the compiler that the function is compiled in C style.

C# with C++

`__declspec(dllexport)`

- It allows the compiler to generate the export names automatically and place them in a .lib file.
- This .lib file can then be used just like a static .lib to link with a DLL.
- You can export data, functions, classes or class member functions from a DLL using it.

C# with C++

```
using System.Runtime.InteropServices;
```

- It provides wide variety of members that support COM interop and platform invoke services.
 - COM interop is a technology included in the .NET Framework Common Language Runtime (CLR) that enables Component Object Model (COM) objects to interact with .NET objects and vice versa (C# is working with .NET).

```
DllImport
```

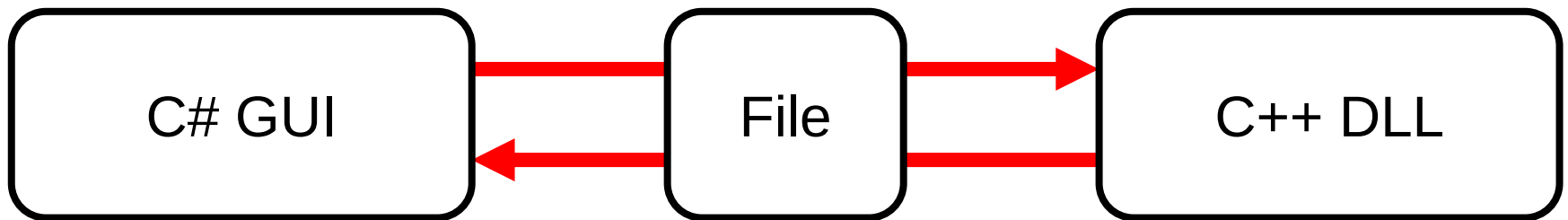
- It is a straightforward way for C# programs to call functions from a DLL file written in C or C++.

```
Int32.Parse(textBox1.Text)
```

- Convert the string to an integer.

C# with C++

- Drawback: it works perfect if the function in DLL returns an integer or a floating point number but not for strings or character arrays.
- Memory allocation problem: DLL programs and C# programs are different and they do not share the same memory space.
- Solution: File processing

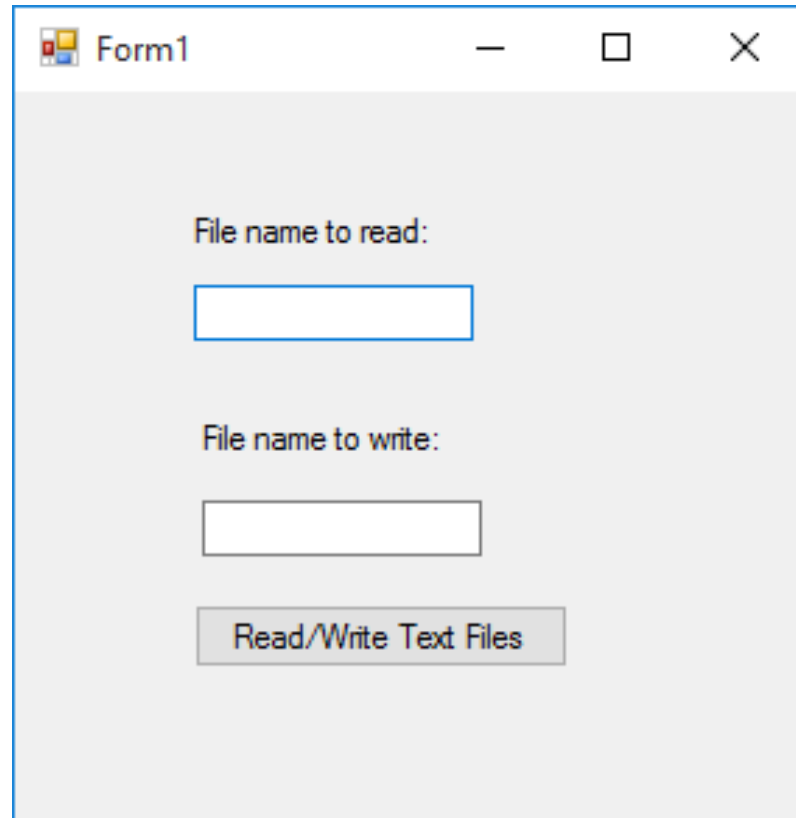


Read/Write Text Files

- Example 4: Read/Write text files
 - Get the file name to read the file content (e.g., a.txt).
 - Get the file name to write the file content (e.g., b.txt).
 - Read the first file (a.txt) and write the content to the second file (b.txt).

Read/Write Text Files

- Step 1: Create a C# GUI program named “ReadWriteTextFiles” and create a form like this:



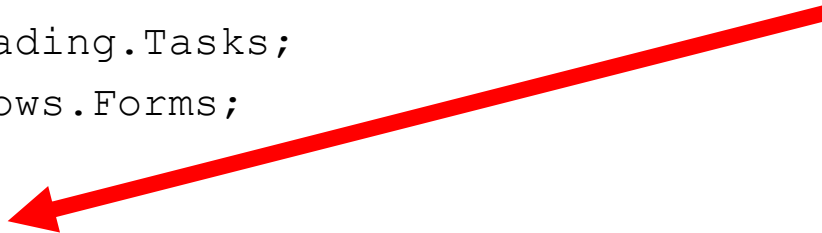
The image shows a screenshot of a Windows Forms application window titled "Form1". The window has a standard Windows title bar with minimize, maximize, and close buttons. The form itself has a light gray background. It contains two text input fields. The first field is preceded by the label "File name to read:". The second field is preceded by the label "File name to write:". Below these two fields is a button with the text "Read/Write Text Files".

Read/Write Text Files

- Step 2: Write the following program code in Form1.cs.

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Drawing;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows.Forms;  
  
using System.IO;
```

New program code



Read/Write Text Files

```
private void button1_Click(object sender, EventArgs e)
{
    String line;
    try
    {
        StreamReader sr = new StreamReader(textBox1.Text);
        StreamWriter sw = new StreamWriter(textBox2.Text);

        line = sr.ReadLine();
        while (line != null)
        {
            MessageBox.Show(line, "File Content",
                MessageBoxButtons.OK);
            sw.WriteLine(line);
            line = sr.ReadLine();
        }
    }
}
```

New
program
code

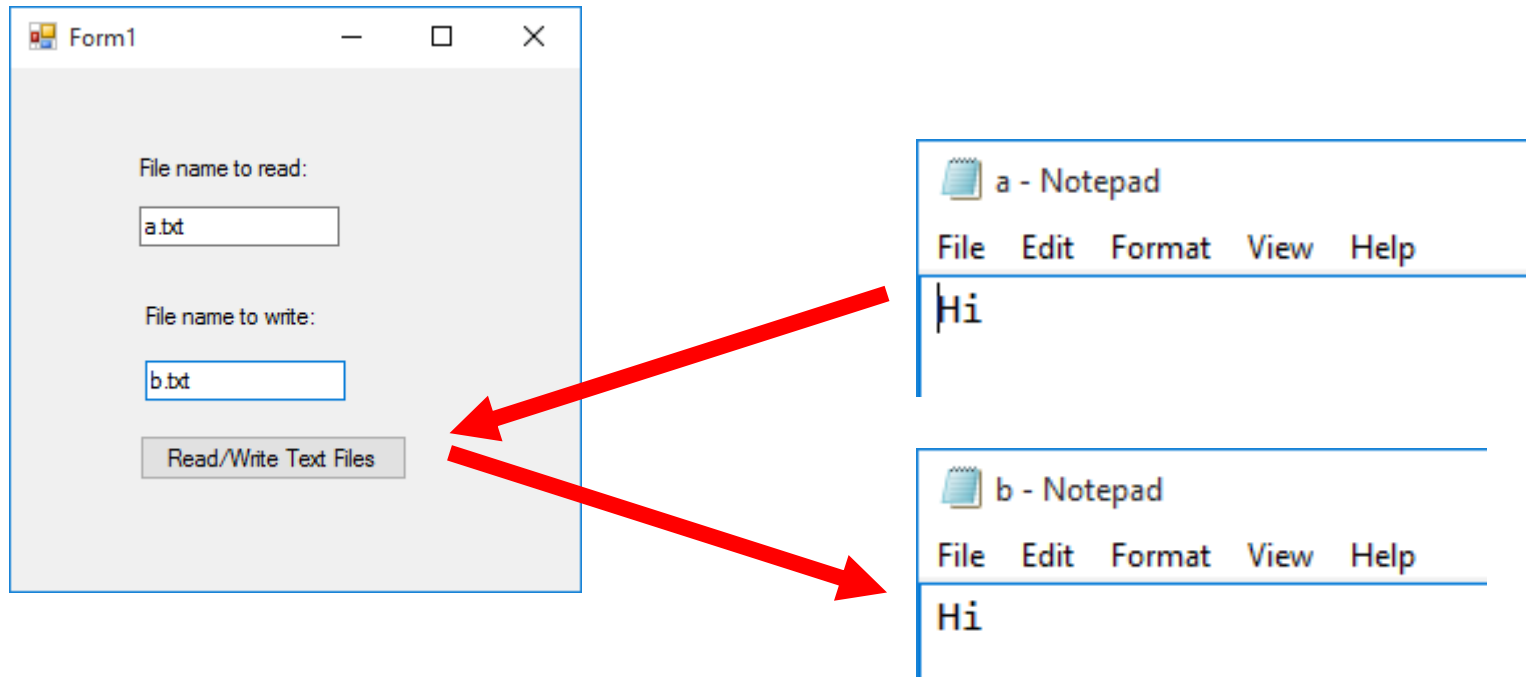
Read/Write Text Files

New
program
code

```
        sr.Close();  
        sw.Close();  
    }  
    finally  
    {  
        MessageBox.Show("Last one", "End",  
                        MessageBoxButtons.OK);  
    }
```

Read/Write Text Files

- Step 3: Copy a.txt into the sub folder (bin/Debug) of the project “ReadWriteTextFiles”.
- Step 4: Compile and run the program.



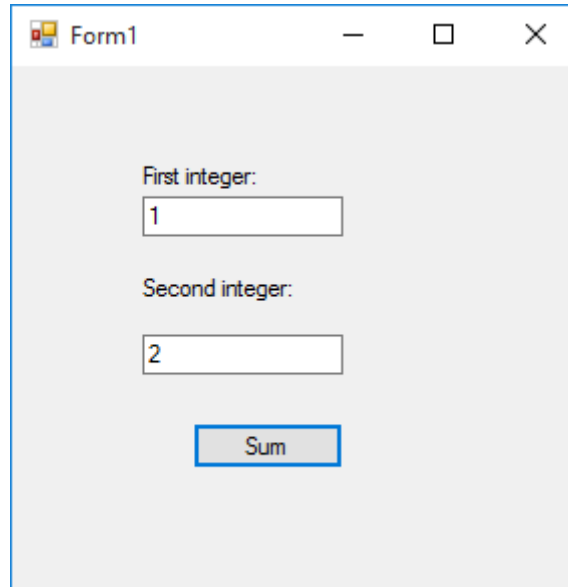
Read/Write Text Files

```
try {...} finally {...}
```

- It tries but it may fail.
- “finally” is to tell the program should do at the end.

C# works with C++ and File Processing

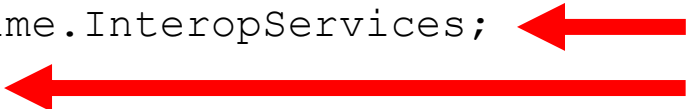
- In the C# GUI program
 - Create your GUI outlook



C# works with C++ and File Processing

- Modify Form1.cs (exactly same as the example).

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Drawing;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows.Forms;  
  
using System.Runtime.InteropServices;  
using System.IO;
```

 New program code

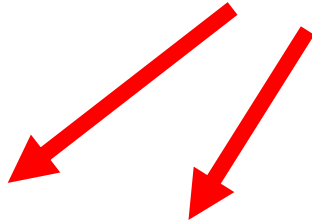
C# works with C++ and File Processing

- Modify Form1.cs (exactly same as the example).

```
namespace GUIExample3
{
    public partial class Form1 : Form
    {
        [DllImport("Example3_DLL.dll")]
        public static extern int doSomething(int a, int b);

        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

New program code



C# works with C++ and File Processing

– Create event handlers

```
private void button1_Click(object sender, EventArgs e)
{
    String line = "";
    StreamWriter sw = new StreamWriter("ToC++.txt");
    ...
    sw.WriteLine(line); // write data to the file
    sw.Close();
    int c = doSomething(2, 3); // 1 function is enough
    // no extra for loop is required because all C++
    // statements should be completed after executing the
    // above statement
    StreamReader sr = new StreamReader("FromC++.txt");
    ...
    line = sr.ReadLine(); // read data from the file
    MessageBox.Show(line, "", MessageBoxButtons.OK);
}
```

New
program
code

C# works with C++ and File Processing

- In the C++ program
 - Create Example3_DLL.cpp

```
#include "Example3_Class.h"
#include <fstream>
using std::ifstream;
using std::ofstream;
using std::ios

extern "C"
{
    __declspec(dllexport) int doSomething(int x, int y)
    {
        ...
    }
}
```

C# works with C++ and File Processing

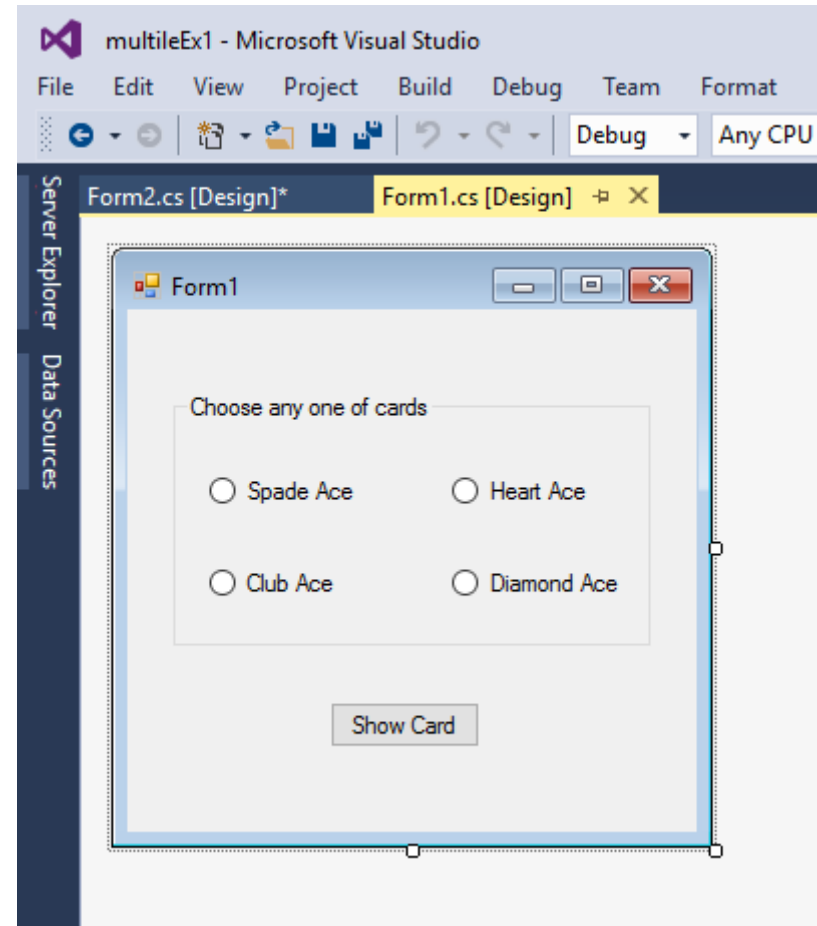
```
if ((x == 2) && (y == 3))
{
    int a;
    ifstream inClientFile("ToC++.txt", ios::in);
    inClientFile >> a; // read data from the file
    ...
    inClientFile.close();
    ... // program code for the corresponding input
    ... // it means you do what you should do
    ... // when x = 2 and y = 3
    ofstream outClientFile("FromC++.txt", ios::out);
    outClientFile << a;
    ...
    outClientFile.close();
}
}
```

Multiple Forms

- There can be more than one windows form in an application.
- Example 5: Select an ace.
 - A user selects one and only one of the four cards.
 - When the user clicks the “Show Card” button, another form (Form2) will be shown with the selected card.
 - The user cannot access the original form (Form1) until he/she clicks the OK button of Form2 (known as the modal form).
 - When the user clicks on the picture, the card will toggle between face up and face down.

Multiple Forms

- Step 1: Create a C# GUI program named “MultipleForm1” and create Form1 like this:
 - Radio button
 - Group box
 - Group radio buttons
 - Put radio buttons inside the group box.
 - Button

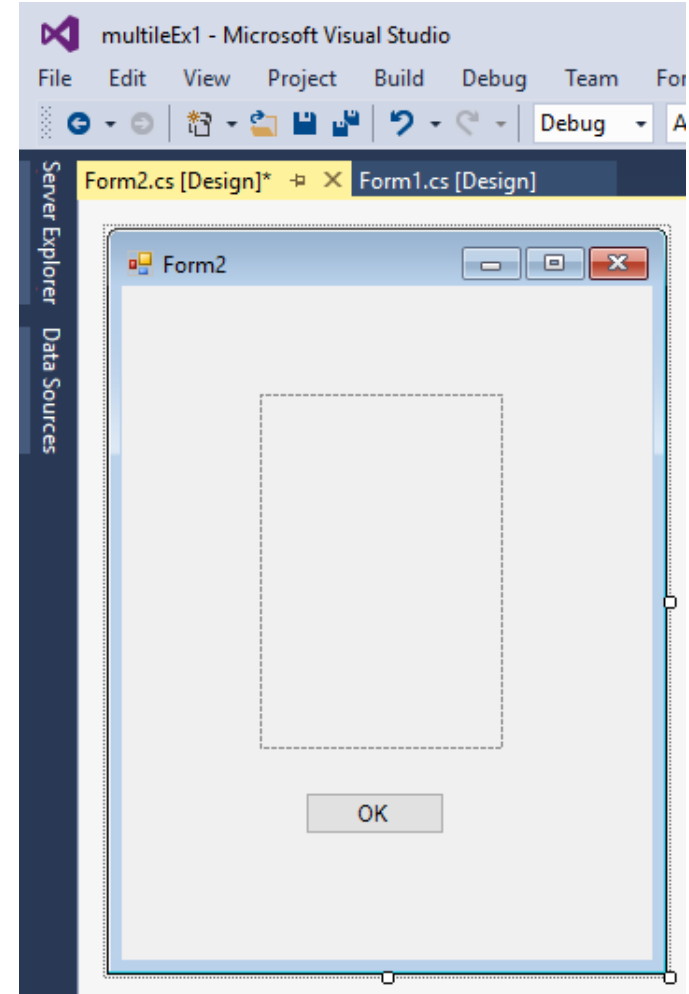


Multiple Forms

– Step 2: Create Form2 like this:

- Picture box
 - Set “SizeMode” from “Normal” to “StretchImage”.
- Change the dimension of the **picture box** to 200x 273.
- Change the dimension of the **form** to 400 x 500.
- Button
 - Go to “Properties”, select “OK” in “**DialogResult**”.
 - It makes the form will be closed automatically after the button “OK” is clicked.

Tip: Alternatively, you may add the click event on the OK button to close this form.
// this.Close();



Multiple Forms

- Step 3: Add the following program code into the function **button1_Click** of Form1.cs:

```
int card = 0;    // 0 = no card, 1 = sA, 2 = hA 3 = cA, 4 = dA
if (radioButton1.Checked) card = 1; // spade ace selected
if (radioButton2.Checked) card = 2; // heart ace selected
if (radioButton3.Checked) card = 3; // club ace selected
if (radioButton4.Checked) card = 4; // diamond ace selected

Form2 F2 = new Form2(card);
F2.ShowDialog();
```

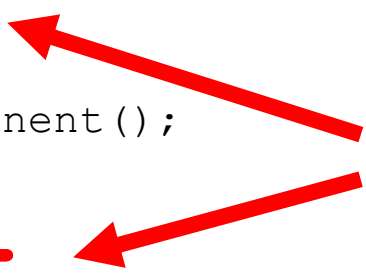
Multiple Forms

- Step 4: Modify the program code of the constructor and add some new variables in Form2.cs:

```
public Form2(int c)
{
    InitializeComponent();
    card = c;
    face = 0;
    ShowCard();
}
```

```
private int card, face;
```

New program code



Multiple Forms

- Step 5: Add one function in Form2.cs just after the constructor and some new variables:

```
void ShowCard()
{
    if (card == 0) pictureBox1.Visible = false;
    if (card == 1)
    {
        pictureBox1.Visible = true;
        pictureBox1.Image = Image.FromFile("images\\s1.jpg");
    }
}
```

Multiple Forms

```
if (card == 2)
{
    pictureBox1.Visible = true;
    pictureBox1.Image = Image.FromFile("images\\h1.jpg");
}
if (card == 3)
{
    pictureBox1.Visible = true;
    pictureBox1.Image = Image.FromFile("images\\c1.jpg");
}
if (card == 4)
{
    pictureBox1.Visible = true;
    pictureBox1.Image = Image.FromFile("images\\d1.jpg");
}
}
```

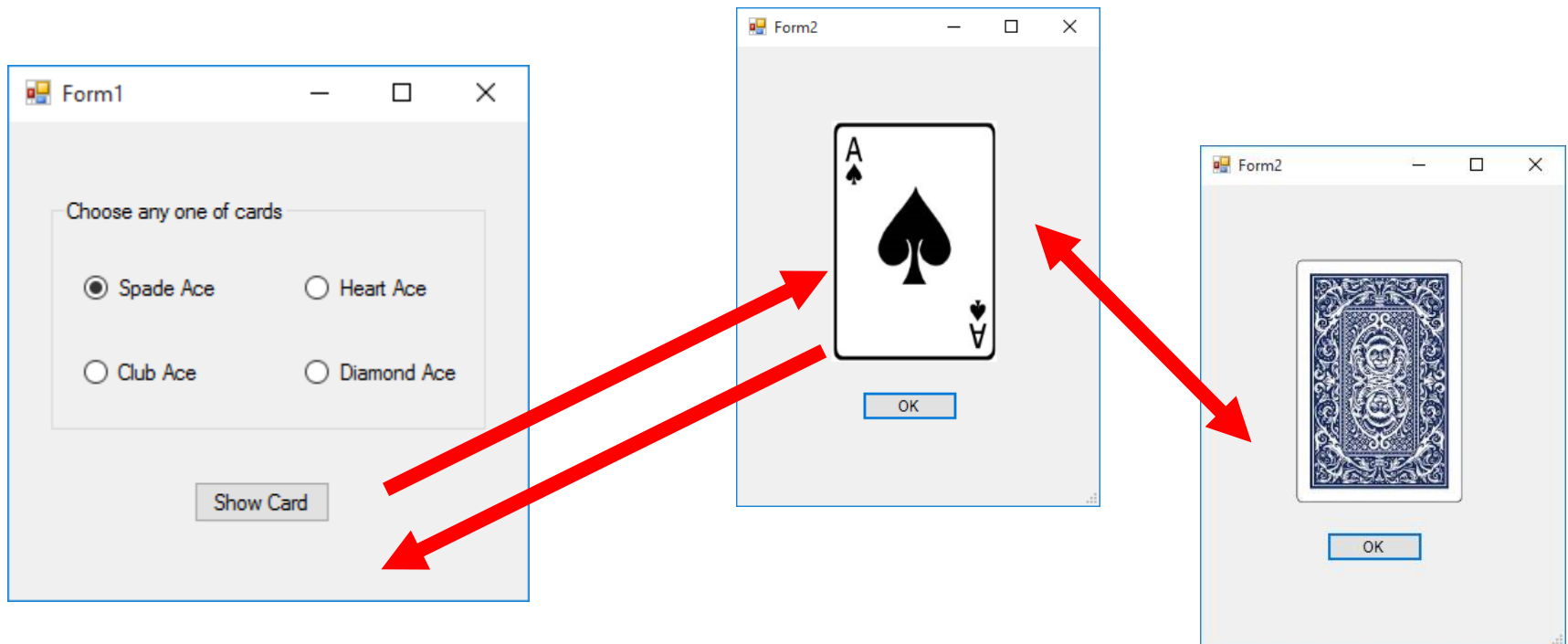
Multiple Forms

- Step 6: Add the following program code into the function **pictureBox1_Click** of Form2.cs:

```
if (card != 0) // card = 0 means no card is selected
{
    if (face == 0) // face = 0 means face-up
    {
        pictureBox1.Image = Image.FromFile("images\\back.jpg");
        face = 1; // facing down the card
    }
    else
    {
        ShowCard();
        face = 0; // currently face-up
    }
}
```

Multiple Forms

- Step 7: Save all image files into a folder “images” and put it into the sub folder (bin/Debug) of the project “MultipleForm1”.
- Step 8: Compile and run the program.



Multiple Forms

```
Form2 F2 = new Form2(card) ;
```

- An object of Form2 is created to generate Form2.
- Since we want to tell Form2 which card is selected, the constructor of Form2 is modified to accept one input parameter that indicates the selected card.

```
F2.ShowDialog() ;
```

- This statement makes Form2 is a modal form.
- It means it cannot access other forms unless we close Form2.
- If Show() is called instead, Form2 is a modales form, which means we can still access other forms without closing it.

Multiple Forms

```
pictureBox1.Visible = false;
```

- Visible is a property of pictureBox1. If it is false, pictureBox1 is not visible.

```
pictureBox1.Image =  
Image.FromFile("images\\s1.jpg");
```

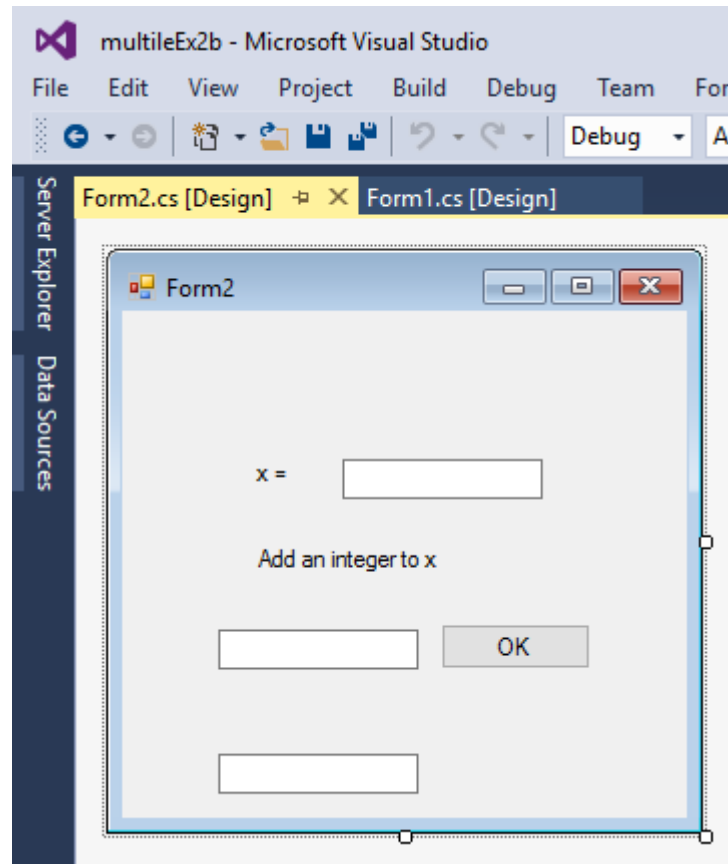
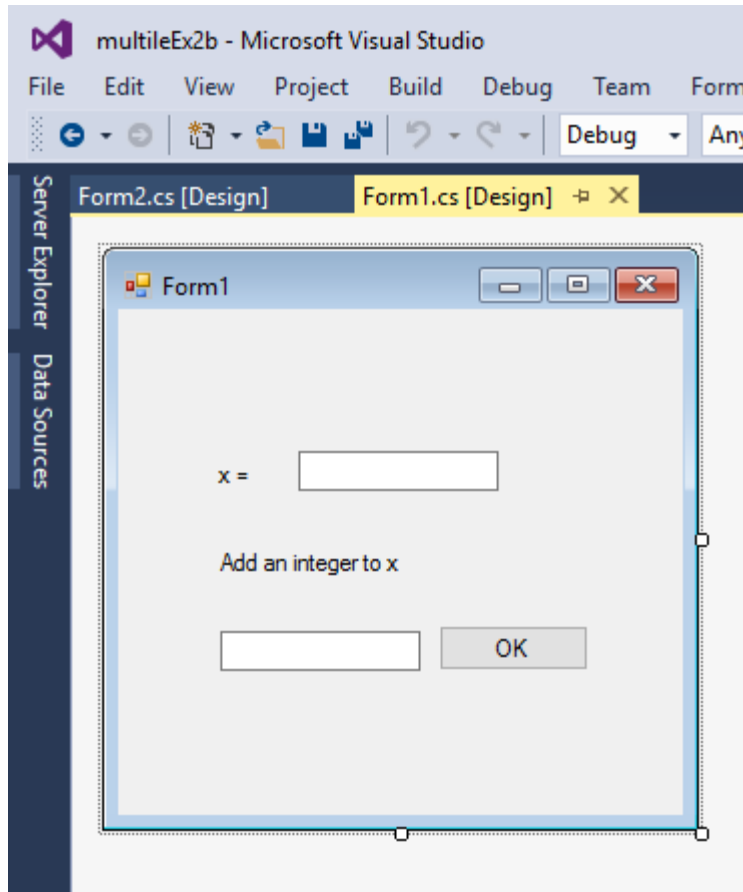
- Image is another property of pictureBox1. It defines the location where the picture can be found.
- FromFile is a static member function of the Image class. It returns a file handler of an image stored at the specified location.
- \ is a special character means escape. If we want to use it as just the \ character, we have to use \\.

Multiple Forms

- Data can be passed between two windows forms.
- Example 6: This example keeps adding an integer to the variable x.

Multiple Forms

- Step 1: Create a C# GUI program named “MultipleForm2” and create two forms like this:



Multiple Forms

- In Form 2, change the setting of Button
 - Go to “Properties”, select “OK” in “**DialogResult**”.
 - It makes the form will be closed automatically after the button “OK” is clicked.

Multiple Forms

- Step 2: Add the following program code into the function **button1_Click** of Form1.cs:

```
int localX;  
localX = 0;  
  
localX = Int32.Parse(textBox1.Text) +  
         Int32.Parse(textBox2.Text);  
Form2 F2 = new Form2(localX);  
F2.ShowDialog();  
textBox1.Text = ""+F2.def();
```

Multiple Forms

- Step 3: Modify the program code of the **constructor** in Form2.cs:

```
public Form2(int a)
{
    InitializeComponent();
    textBox1.Text = "" + a;
}
```

New program code

Multiple Forms

- Step 4: Add one function and one variable in Form2.cs just after the `constructor`:

```
public int values;  
  
public int def()  
{  
    return values;  
}
```

Multiple Forms

- Step 5: Add the following program code into the function **button1_Click** of Form2.cs:

```
values = System.Convert.ToInt32(textBox1.Text) +  
        System.Convert.ToInt32(textBox2.Text);  
textBox3.Text = "" + values;
```

Multiple Forms

- Step 6: Compile and run the program.

The image displays three screenshots of a Windows application with two forms, Form1 and Form2, illustrating a sequence of operations. Red arrows indicate the flow of execution between the forms.

Form1 (Left): The variable `x` is 22. The input field for "Add an integer to x" contains 32. The OK button is disabled.

Form2 (Bottom): The variable `x` is 54. The input field for "Add an integer to x" contains 378. The OK button is disabled.

Form1 (Right): The variable `x` is 432. The input field for "Add an integer to x" contains 32. The OK button is disabled.

The sequence of operations is as follows:

- Form1 (Left) is shown with `x = 22` and input 32.
- A red arrow points from Form1 (Left) to Form2 (Bottom).
- Form2 (Bottom) is shown with `x = 54` and input 378.
- A red arrow points from Form2 (Bottom) to Form1 (Right).
- Form1 (Right) is shown with `x = 432` and input 32.

GDI

- Using the .NET framework class library's Windows Forms Controls is not the only way to graphically present data.
- Windows Forms Controls sometimes are too restrictive that introduce difficulties in displaying graphic data. For example,
 - To draw a line on the screen
 - To put an image on a button
 - To show a sequence of images as an animation, etc.

GDI

- Graphics can be put into a program only if a related control is found, hence limits creativity.
- GDI+ originated from the Windows Graphical Device Interface (GDI) with several improvements.
- To programmers, GDI+ is just a set of namespaces that provide library functions for the rendering of 2D graphics. It provides supports for colors, pens, fonts, image transformations, etc.

GDI

- GDI+ is device independent – output of the graphics need not be shown on the screen, can be on the printer.
- When starting a Windows Form project in Visual Studio, GDI+ has been included by default.
- For detailed discussion of GDI+, one can refer to Microsoft home page:

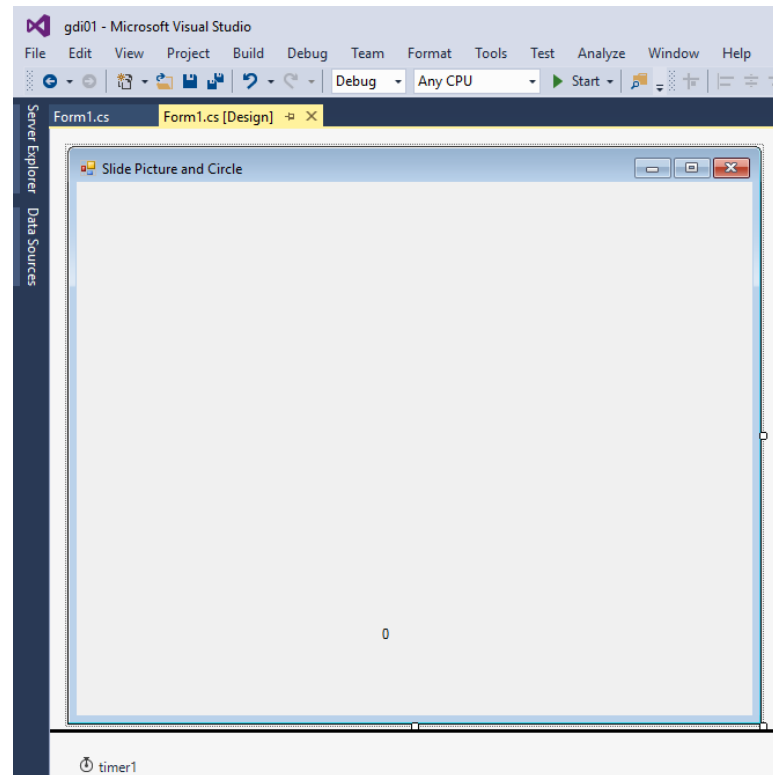
<https://docs.microsoft.com/en-us/windows/win32/gdiplus/-gdiplus-using-gdi--use>

GDI

- Example 7: moving objects
 - The card will fly from left to right and the circle will fly from right to left.
 - A counter is put at the bottom to show the current horizontal position of the circle.
 - When clicking on the counter, the motion of both circle and card will stop.
 - When clicking on the counter again, the motion restarts.

GDI

- Step 1: Create a C# GUI program named “GDI” and create a form like this:
 - Size: 600 x 500
 - Text: “Slide Picture and Circle”
 - Add a label.
 - Add a timer.



GDI

- Step 2: Right click the “timer1” then, “Enable” the timer and change interval to “10” (10 ms).
- Step 3: Right click the form, then select “Properties” and click “Event” and change Paint to “Form1_Paint”.

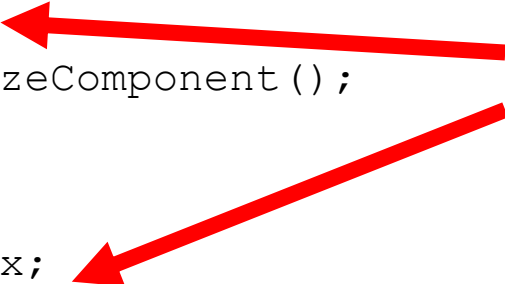
GDI

- Step 4: Modify the constructor and add one new variable just after the constructor:

```
public Form1()  
{  
    x = 0;  
    InitializeComponent();  
}
```

```
private int x;
```

New program code



GDI

- Step 5: Double click “timer1” and you will see the function Timer1_Click. Add the following program code into it:

```
private void timer1_Tick(object sender, EventArgs e)
{
    if (x % 10 == 0) // If X is a multiple of 10,
    {
        label1.Text = "" + x; // show its value on label1
    }
    this.Invalidate();
}
```

GDI

- Step 6: Double click “label1” and you will see the function label1_Click. Add the following program code into it:

```
private void label1_Click(object sender, EventArgs e)
{
    timer1.Enabled = !(timer1.Enabled);
}
```

GDI

- Step 7: Double click “Form1” and you will see the function Form1_Load. Add the following program code into it:

```
private void Form1_Load(object sender, EventArgs e)
{
    this.DoubleBuffered = true;
    this.Paint += Form1_Paint;
}
```

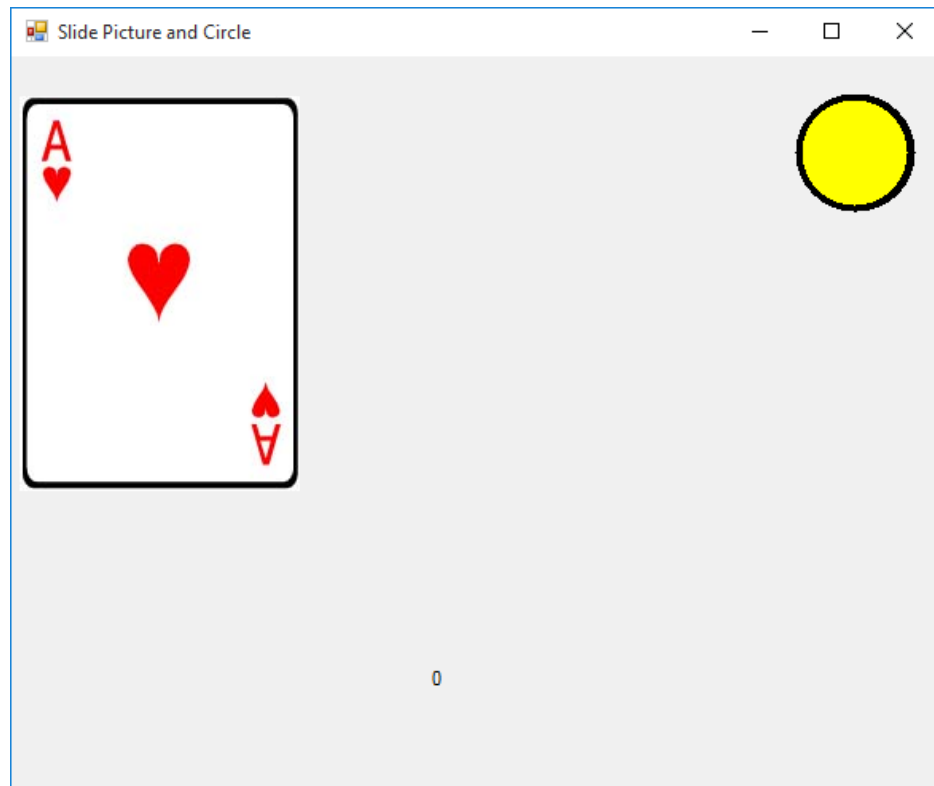
GDI

- Step 7: Double click “Form1” and you will see the function Form1_Paint. Add the following program code into it:

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    if (x < this.Size.Width - 1) { x += 1; }
    else { x = 0; }
    Bitmap bp = new Bitmap(Image.FromFile("images\\h1.jpg"));
    g.DrawImage(bp, x, 25);
    Rectangle Head = new Rectangle(this.Size.Width - 100 - x,
                                   25, 70, 70);
    g.FillEllipse(Brushes.Yellow, Head);
    Pen b4pen = new Pen(Color.Black, 4);
    g.DrawEllipse(b4pen, Head);
}
```

GDI

- Step 8: Save all image files into a folder “images” and put it into the sub folder (bin/Debug) of the project “GDI”.
- Step 9: Compile and run the program.



GDI

Timer

- A timer is one of the .NET GUI controls.
- When enabled, it generates an event for every clock tick, by default 100 msec.
- An event handler can be designed in response to every clock tick, hence useful to the tasks that need to be done regularly in time, e.g., to show an animation.
- By default, timers are disabled.
- All defaults can be changed in the property window.

GDI

Paint

- All controls generate a Paint event when they determine that they need to be updated on their shape.
- More specifically, a Paint event is triggered whenever
 - a control is created, resized, or restored; or
 - when another control that had overlaid it is moved.
- By default, the Paint event of a control is handled by one of its member functions called OnPaint().
- Additional event handler can be added if one wants to do additional thing whenever the control is updated.

GDI

```
this.Invalidate();
```

- Invalidate() method is a manual way of triggering a Paint event.
- Since we are calling the Invalidate() of Form1, hence the Paint event handler of Form1 will be called.
- This is equivalent to Invalidate().
- Just to indicate we are not talking about other Invalidate(), but the Invalidate() of Form1.

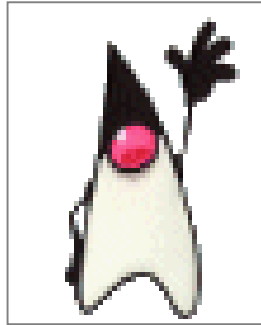
GDI

```
this.DoubleBuffered = true;
```

- Rendering graphic information and transfer them onto the screen can be a relatively slow process.
- Graphic is rendered pixel-by-pixel directly onto the screen.
- It so happens only half of the picture is drawn when it needs to be displayed.
- When using double buffer, graphic is first rendered into a buffer (memory first).
- When the rendering process finishes, the whole buffer data are transferred to the screen in a block.
- Result in much faster and smoother graphic and image drawing.

GDI

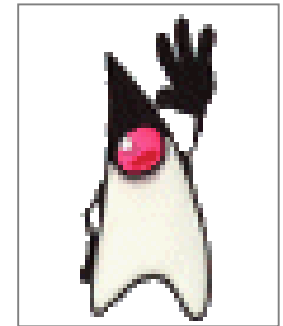
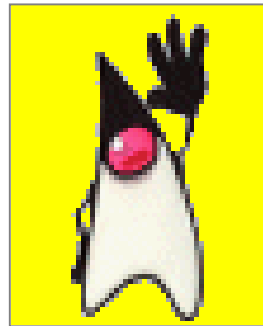
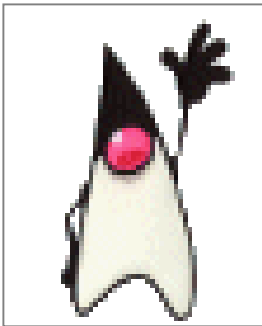
- Pixel by pixel



Traditional way

Screen

- Block by block



**The original
graphic**

Undisplayed Buffer

Screen

GDI

```
this.Paint += Form1_Paint;
```

- Add an event handler to the Paint event.
- This statement is to call the function Form1_Paint when the Paint event is triggered.

```
Graphics g = e.Graphics;
```

- The Graphics class is the heart of all rendering activity of GDI+.
- It is a device-independent representation of the drawing surface that you plan to render graphics on.
- The above statement indicates that g is a Graphic object, is created on Form1.

GDI

```
if (x < this.Size.Width - 1) { x += 1; }  
    else { x = 0; }
```

- For each Paint event, x is advanced by 1.
- It goes back to 0 when it is equal to the width of Form1.
- Since Form1 can be resized during run time, we need to check the actual width of Form1 in the program.
- It is done by this.Size.Width.

GDI

```
Bitmap bp = new  
    Bitmap(Image.FromFile("images\\h1.jpg"));  
g.DrawImage(bp, x, 25);
```

- The class Bitmap is for the temporary storage of image data.
- The constructor of Bitmap requires a string that indicates the path of the image file.
- The location of the Bitmap created is indicated by a pointer bp.
- We can render the data in the Bitmap on the drawing surface g using the DrawImage function.
- The 2nd and 3rd parameters of DrawImage refer to the X and Y coordinates of the upper left corner at which the image is drawn.

GDI

Rectangle Head =

```
new Rectangle(this.Size.Width - 100 - x, 25, 70, 70);
```

- The class Rectangle is a subclass of Drawing.
 - 1st and 2nd – the X and Y coordinates of the upper left corner of the rectangle
 - 3rd and 4th – the length and the width of the rectangle

```
g.FillEllipse(Brushes.Yellow, Head);
```

- Create a filled ellipse.
- The ellipse is filled using the Yellow brush and has size the same as that defined by the rectangular region Head.

GDI

```
Pen b4pen = new Pen(Color.Black, 4);
```

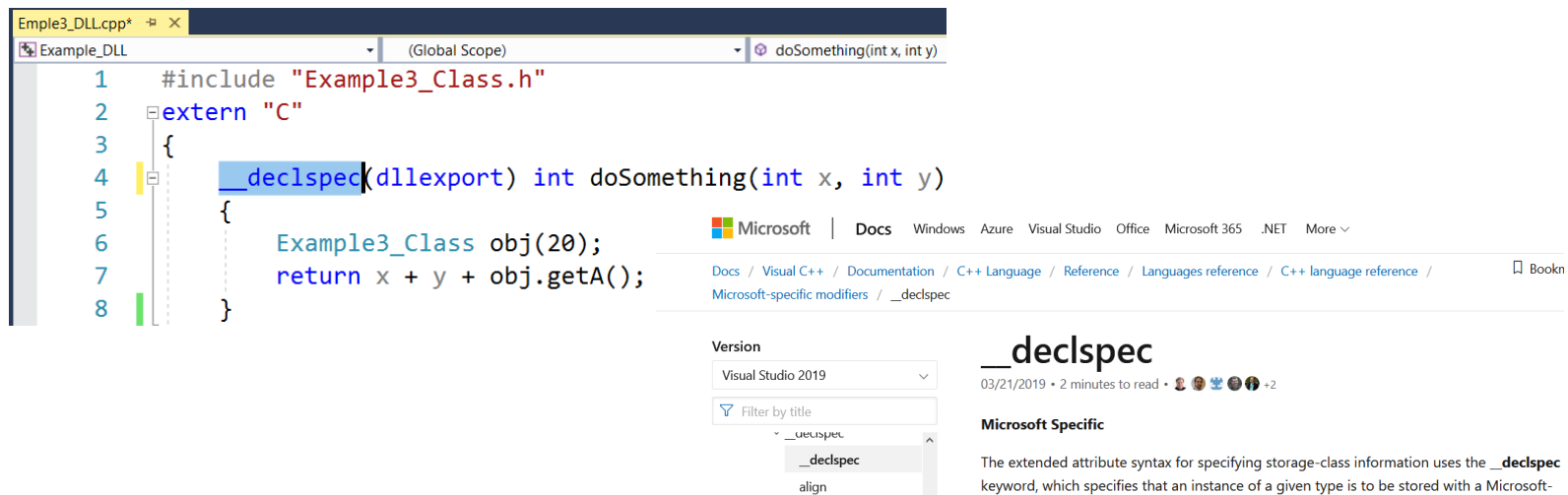
- Besides brush, GDI+ provides the Pen class for one to draw a graphic structure.
- The class Pen lets us define the pen we can use for drawing.
 - 1st parameter – the color of the pen
 - 2nd parameter – the size of the pen

```
g.DrawEllipse(b4pen, Head);
```

- Enable one to draw a circle on the drawing surface g with size as specified in Head using the pen b4pen.

Useful Tips

- Quick help
 - One quick way to search for help online or to learn more from Microsoft is to highlight the keyword in your source code first and then press F1. (The description is for VS2019.)



The screenshot shows the Visual Studio 2019 interface. In the code editor, the file `Emple3_DLL.cpp` is open, and the `doSomething(int x, int y)` function is selected. The code includes `Example3_Class.h` and defines the function using `__declspec(dllexport)`. The help window on the right displays the `__declspec` keyword, including its version (Visual Studio 2019), a filter by title, and a list of related keywords like `__declspec` and `align`. The `Microsoft Specific` section explains that `__declspec` is used for specifying storage-class information.

```
1 #include "Example3_Class.h"
2 extern "C"
3 {
4     __declspec(dllexport) int doSomething(int x, int y)
5     {
6         Example3_Class obj(20);
7         return x + y + obj.getA();
8     }
```

Microsoft | Docs Windows Azure Visual Studio Office Microsoft 365 .NET More

Docs / Visual C++ / Documentation / C++ Language / Reference / Languages reference / C++ language reference / Microsoft-specific modifiers / __declspec

Version: Visual Studio 2019

Filter by title: __declspec, align

__declspec

03/21/2019 • 2 minutes to read • 1 2 3 4 +2

Microsoft Specific

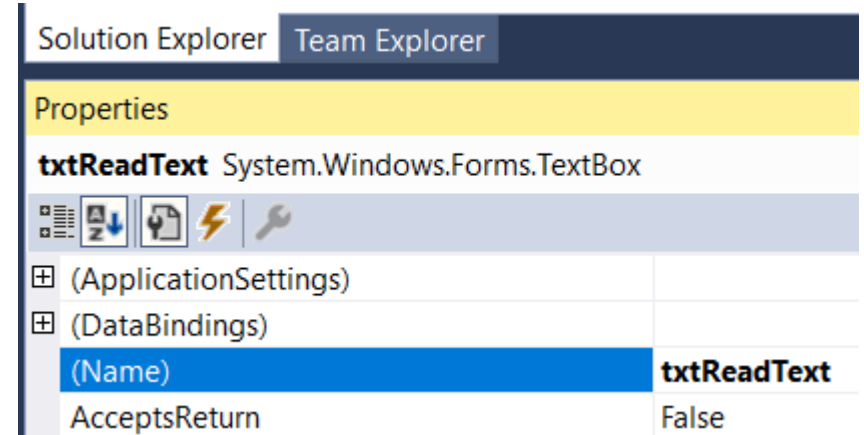
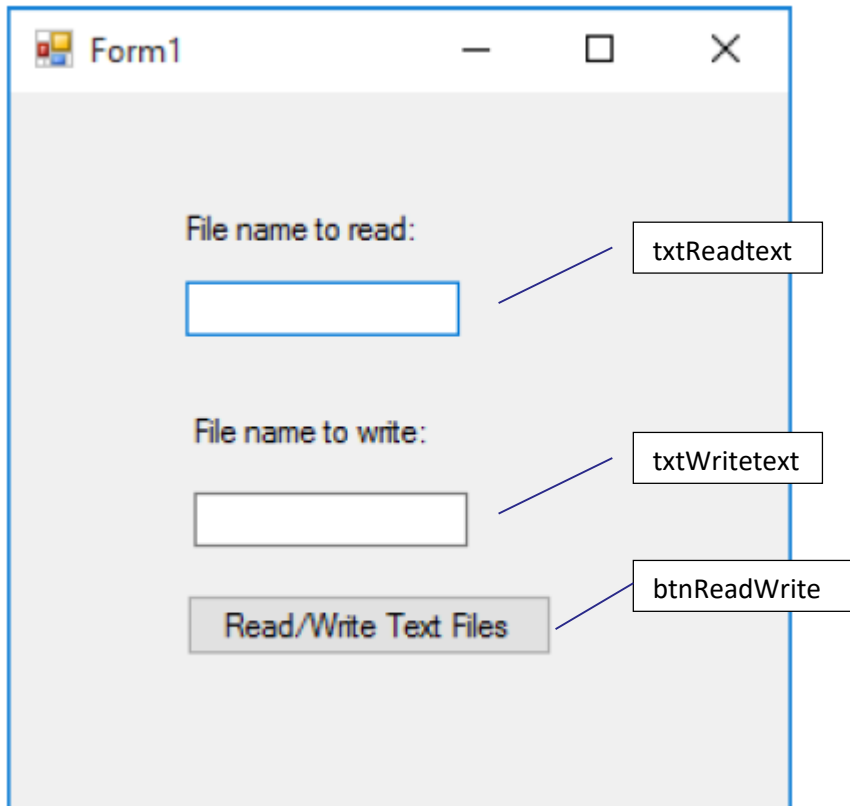
The extended attribute syntax for specifying storage-class information uses the `__declspec` keyword, which specifies that an instance of a given type is to be stored with a Microsoft-

Useful Tips

- Good naming convention
 - One of the important property of each object is **“(Name)”**
 - You may change the default name to a proper naming convention such as **txt**FullName for textbox or **btn**Submit for button. After this extra work, you can easily manipulate your programming such as

```
txtFullName.text = txtSurName.text + " " + txtFirstName.text;
```

Useful Tips



Reference Readings

- <https://www.microsoft.com>
- Wikipedia

End